

Guia de Estudo — Sistema de Gerenciamento de Tarefas

Leia este documento do início ao fim. Cada arquivo do projeto é explicado com o **quê**, o **porquê** e o **como**.

Índice

1. [Visão Geral da Arquitetura](#)
 2. [Backend — Arquivo por Arquivo](#)
 3. [Frontend — Arquivo por Arquivo](#)
 4. [Docker — Como Tudo se Conecta](#)
 5. [CI/CD — GitHub Actions](#)
 6. [Fluxo Completo de uma Requisição](#)
-

1. Visão Geral da Arquitetura

Este projeto é uma aplicação **Full-Stack**, dividida em três camadas:

```
[ Navegador (React) ]  
  ⚡ HTTP (Axios)  
[ Servidor (Node.js + Express) ]  
  ⚡ SQL (pg)  
[ Banco de Dados (PostgreSQL) ]
```

Por que essa separação?

- O **frontend** só sabe mostrar dados e capturar cliques do usuário
- O **backend** contém toda a lógica de negócio e regras de segurança
- O **banco de dados** é o único responsável por guardar informações permanentemente

Se o frontend fosse direto ao banco, qualquer pessoa poderia ver os dados de outros usuários. O backend age como um "porteiro" que valida quem pode acessar o quê.

2. Backend — Arquivo por Arquivo

`backend/.env`

```
PORT=3000  
JWT_SECRET=minha_chave_secreta_super_segura_para_dev  
DB_USER=meu_usuario  
DB_HOST=localhost  
DB_NAME=gerenciador_tarefas
```

```
DB_PASSWORD=minha_senha_secreta
DB_PORT=5432
```

O que é: Arquivo de variáveis de ambiente. Nunca vai para o GitHub (está no `.gitignore`).

Por que existe: Separar configuração do código. Em produção, você troca só o `.env` — o código não muda. É uma boa prática de segurança: nenhuma senha aparece no código.

Regra de ouro: Se você ver `process.env.ALGO` no código, o valor vem daqui.

backend/src/database.js

```
const { Pool } = require('pg');

const pool = new Pool({
  user: process.env.DB_USER,
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  password: process.env.DB_PASSWORD,
  port: process.env.DB_PORT,
});
```

O que é: Configura a conexão com o PostgreSQL.

O que é um Pool? Em vez de abrir e fechar uma conexão a cada requisição (lento), o Pool mantém várias conexões abertas e as reutiliza. Quando chega uma requisição, pega uma conexão disponível do "pool", usa, e devolve.

`pool.query(sql, [params])` — é assim que você executa SQL no projeto inteiro. O segundo parâmetro é uma lista de valores que substituem os `$1, $2...` na query, evitando **SQL Injection**.

backend/src/criarTabelas.js

```
CREATE TABLE IF NOT EXISTS users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(255) NOT NULL,
  "passwordHash" VARCHAR(255) NOT NULL,
  "createdAt" TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS tasks (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  "userId" UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  completed BOOLEAN DEFAULT false,
```

```
"dueDate" DATE,  
"createdAt" TIMESTAMPT DEFAULT CURRENT_TIMESTAMP,  
"updatedAt" TIMESTAMPT DEFAULT CURRENT_TIMESTAMP  
);
```

O que é: Cria as tabelas no banco ao iniciar o servidor.

IF NOT EXISTS — garante que rode toda vez sem dar erro se as tabelas já existem.

UUID — identificador único universal. Preferido ao **id SERIAL** (1, 2, 3...) porque não vaza informação (um atacante não consegue adivinhar IDs de outros usuários).

REFERENCES users(id) ON DELETE CASCADE — chave estrangeira. Cada tarefa pertence a um usuário. Se o usuário for deletado, todas as tarefas dele são deletadas automaticamente.

passwordHash — nunca guardamos a senha em texto puro. Guardamos o hash gerado pelo bcrypt.

backend/src/index.js

```
require('dotenv').config(); // 1. Carrega o .env  
const express = require('express');  
const cors = require('cors');  
// ...importações...  
  
app.use(cors()); // 2. Permite requisições do frontend  
app.use(express.json()); // 3. Lê o corpo das requisições em JSON  
  
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec)); // 4. Docs  
app.use('/api/auth', authRoutes); // 5. Rotas de autenticação  
app.use('/api/tasks', taskRoutes); // 6. Rotas de tarefas
```

O que é: Ponto de entrada do servidor. Onde tudo começa.

CORS — Cross-Origin Resource Sharing. Por padrão, o navegador bloqueia requisições de **localhost:5173** para **localhost:3000** porque são "origens diferentes". O **cors()** desabilita essa restrição.

app.use(express.json()) — sem isso, **req.body** fica vazio. Este middleware lê o corpo da requisição HTTP e converte o JSON em objeto JavaScript.

require.main === module — técnica para que o servidor só inicie quando o arquivo é executado diretamente (**node src/index.js**), e não quando é importado nos testes.

backend/src/middlewares/authMiddleware.js

```
const verificarToken = (req, res, next) => {  
  const authHeader = req.headers['authorization'];  
  const token = authHeader && authHeader.split(' ')[1]; // "Bearer TOKEN" →
```

```
"TOKEN"

if (!token) return res.status(401).json({ error: 'Token não fornecido.' });

try {
  const verificado = jwt.verify(token, process.env.JWT_SECRET);
  req.userId = verificado.id; // injeta o ID do usuário na requisição
  next();                    // passa para o próximo handler
} catch (error) {
  return res.status(403).json({ error: 'Token inválido ou expirado.' });
}
};
```

O que é: Um "porteiro" que protege rotas. Roda antes do controller.

Middleware — função que fica no meio do caminho entre a requisição e a resposta. Recebe (`req`, `res`, `next`). Chama `next()` para deixar passar, ou `res.json()` para barrar.

Como o token chega: O frontend envia no cabeçalho `Authorization: Bearer eyJhbGc...`

`req.userId = verificado.id` — truque importante: injeta o ID do usuário autenticado diretamente no objeto da requisição, para que o controller saiba de quem é sem precisar decodificar o token de novo.

Códigos HTTP:

- `401 Unauthorized` — não enviou token
- `403 Forbidden` — enviou token, mas é inválido ou expirou

backend/src/routes/authRoutes.js e taskRoutes.js

```
// authRoutes.js
router.post('/register', authController.register);
router.post('/login', authController.login);

// taskRoutes.js
router.get('/',          verificarToken, taskController.list);
router.post('/',         verificarToken, taskController.create);
router.patch('/:id',     verificarToken, taskController.update);
router.patch('/:id/complete', verificarToken, taskController.complete);
router.delete('/:id',    verificarToken, taskController.delete);
```

O que são: Mapeamento de URL + método HTTP → função que responde.

`/:id` — parâmetro dinâmico. Se a URL for `/tasks/abc-123`, então `req.params.id` vale `"abc-123"`.

verificarToken no meio — middleware aplicado só nessa rota. As rotas de auth não precisam de token (você ainda não está logado). As rotas de tarefas precisam.

Ordem importa: `router.patch('/:id/complete', ...)` deve vir antes de qualquer rota genérica `/:id`, senão o Express interpreta `complete` como um ID.

backend/src/controllers/authController.js

```
const register = async (req, res) => {
  const { name, email, password } = req.body;

  if (!name || !email || !password)
    return res.status(400).json({ error: 'Preencha todos os campos.' });

  try {
    const user = await userService.registarUtilizador(name, email, password);
    return res.status(201).json({ message: 'Registado!', user });
  } catch (error) {
    return res.status(400).json({ error: error.message });
  }
};
```

O que é: Recebe a requisição HTTP, valida os dados e chama o Service.

Responsabilidade única: O controller NÃO faz queries no banco. Ele só valida a entrada e delega para o Service. Isso facilita testes e manutenção.

Códigos HTTP:

- **400 Bad Request** — dados inválidos ou email já existe
 - **201 Created** — recurso criado com sucesso
 - **200 OK** — operação bem-sucedida
 - **401 Unauthorized** — credenciais erradas no login
-

backend/src/controllers/taskController.js

```
const list = async (req, res) => {
  const tasks = await taskService.listarTarefas(req.userId);
  return res.status(200).json({ tasks });
};

const create = async (req, res) => {
  const { title, description, dueDate } = req.body;
  if (!title) return res.status(400).json({ error: 'Título obrigatório.' });
  const task = await taskService.criarTarefa(title, description, dueDate, req.userId);
  return res.status(201).json({ task });
};
```

Ponto chave: `req.userId` — esse valor foi injetado pelo `authMiddleware`. O controller confia nele para saber qual usuário está fazendo a requisição. Isso garante que cada usuário só vê e altera suas próprias tarefas.

backend/src/services/userService.js

```
const registrarUtilizador = async (name, email, password) => {
  // 1. Verifica se email já existe
  const existe = await pool.query('SELECT * FROM users WHERE email = $1',
    [email]);
  if (existe.rows.length > 0) throw new Error('E-mail já está em uso.');
```

// 2. Gera o hash da senha (nunca salva a senha em texto puro!)

```
const salt = await bcrypt.genSalt(10);
const passwordHash = await bcrypt.hash(password, salt);

// 3. Insere no banco e retorna o usuário (sem a senha)
const result = await pool.query(
  `INSERT INTO users (name, email, "passwordHash") VALUES ($1, $2, $3)
  RETURNING id, name, email, "createdAt"`,
  [name, email, passwordHash]
);
return result.rows[0];
};
```

O que é bcrypt: Algoritmo de hash para senhas. Diferente do MD5/SHA, é lento por design — isso dificulta ataques de força bruta. O `salt` é um valor aleatório adicionado antes do hash para que duas senhas iguais gerem hashes diferentes.

RETURNING — cláusula do PostgreSQL que retorna os dados inseridos sem precisar fazer um SELECT separado. Retornamos tudo exceto `passwordHash` — nunca enviamos a senha para o cliente.

backend/src/services/taskService.js

```
const listarTarefas = async (userId) => {
  const result = await pool.query(
    `SELECT * FROM tasks WHERE "userId" = $1 ORDER BY "createdAt" DESC`,
    [userId]
  );
  return result.rows;
};

const criarTarefa = async (title, description, dueDate, userId) => {
  const result = await pool.query(
    `INSERT INTO tasks (title, description, "dueDate", "userId")
    VALUES ($1, $2, $3, $4) RETURNING *`,
    [title, description || null, dueDate || null, userId]
  );
};
```

```
    return result.rows[0];
  };

  const completarTarefa = async (id, userId) => {
    const result = await pool.query(
      `UPDATE tasks SET completed = true, "updatedAt" = NOW()
       WHERE id = $1 AND "userId" = $2 RETURNING *`,
      [id, userId]
    );
    if (result.rows.length === 0) throw new Error('Tarefa não encontrada.');
```

Segurança em toda query: Note que toda query de tarefa inclui `AND "userId" = $x`. Isso garante que um usuário nunca consiga acessar tarefas de outro, mesmo que descubra o ID.

description || null — se não veio descrição, salva NULL no banco (não string vazia).

completarTarefa com verificação: Se o `UPDATE` não afetou nenhuma linha (tarefa não existe ou pertence a outro usuário), `result.rows` fica vazio e lançamos um erro.

backend/src/docs/swagger.js

```
const swaggerSpec = swaggerJsdoc({
  definition: {
    openapi: '3.0.0',
    info: { title: 'Tasks API', version: '1.0.0' },
    components: {
      securitySchemes: {
        bearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT' }
      }
    }
  },
  apis: ['./src/routes/*.js'], // lê os comentários JSDoc das rotas
});
```

O que é: Gera automaticamente a documentação da API em `/api-docs` lendo comentários especiais (`@swagger`) nos arquivos de rotas.

Por que é útil: Qualquer desenvolvedor (ou você mesmo daqui 6 meses) pode abrir `/api-docs` e entender exatamente o que a API faz, quais parâmetros aceita e o que retorna — sem precisar ler o código.

backend/src/__tests__/auth.test.js e tasks.test.js

```
describe('POST /api/auth/register', () => {
  it('deve registrar um utilizador com sucesso', async () => {
    const res = await request(app)
```

```
.post('/api/auth/register')
.send({ name: 'Teste', email: 'teste@teste.com', password: '123456' });

expect(res.status).toBe(201);
expect(res.body).toHaveProperty('token');
});
});
```

O que é: Testes de integração com Jest + Supertest.

Supertest — simula requisições HTTP sem precisar do servidor rodando de verdade. Usa a instância **app** diretamente.

describe — agrupa testes relacionados. **it** — descreve um comportamento esperado. **expect** — a asserção: "espero que isso seja verdade".

Por que testar? Garante que quando você mexer no código amanhã, nada que funcionava vai quebrar sem você saber.

3. Frontend — Arquivo por Arquivo

frontend/src/types/index.ts

```
export interface Task {
  id: string;
  title: string;
  description?: string;
  completed: boolean;
  dueDate?: string;
  userId: string;
  createdAt: string;
  updatedAt: string;
}
```

O que é: Definições de tipos TypeScript compartilhados por todo o frontend.

Por que TypeScript? Quando você escreve **task.ttle** por engano (erro de digitação), o TypeScript avisa antes de você rodar o código. Em JavaScript puro, você só descobriria o erro no navegador.

? após o nome — campo opcional. **description?** significa que pode ser **undefined**.

frontend/src/services/api.ts

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:3000/api',
});
```

```
// Interceptor de REQUISIÇÃO: injeta o token em toda chamada
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Interceptor de RESPOSTA: redireciona ao login se token expirar
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401 || error.response?.status === 403) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

O que é: Instância configurada do Axios com interceptors automáticos.

`import.meta.env.VITE_API_URL` — variável de ambiente do Vite (lida do arquivo `.env`). Prefixo `VITE_` é obrigatório para que o Vite a exponha para o código do browser.

Interceptors — funções que rodam automaticamente antes de toda requisição (request) ou depois de toda resposta (response). É como um middleware, mas no frontend.

Benefício: Você nunca precisa lembrar de colocar o token manualmente em cada chamada — o interceptor faz isso automaticamente.

frontend/src/services/authService.ts

```
export const authService = {
  async login(email, password) {
    const { data } = await api.post('/auth/login', { email, password });
    return data; // { user, token }
  },

  saveSession(token, user) {
    localStorage.setItem('token', token);
    localStorage.setItem('user', JSON.stringify(user));
  },

  isAuthenticated() {
    return !!localStorage.getItem('token'); // !! converte para boolean
  },
};
```

O que é: Encapsula toda a lógica de autenticação no frontend.

localStorage — armazenamento persistente no navegador. Os dados ficam mesmo após fechar a aba. Diferente do **sessionStorage** que apaga ao fechar.

!! — converte qualquer valor para boolean. `!!null = false`, `!!"texto" = true`.

frontend/src/context/AuthContext.tsx

```
export const AuthContext = createContext<AuthContextType | null>(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState<User | null>(null);

  // Ao carregar a página, verifica se já tem sessão salva
  useEffect(() => {
    const savedUser = authService.getUser();
    if (savedUser && authService.isAuthenticated()) setUser(savedUser);
  }, []);

  const login = async (email, password) => {
    const response = await authService.login(email, password);
    authService.saveSession(response.token, response.user);
    setUser(response.user);
  };

  return (
    <AuthContext.Provider value={{ user, isAuthenticated: !!user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  return useContext(AuthContext);
}
```

O que é: Estado global de autenticação acessível em qualquer componente.

Context API — solução do React para compartilhar dados entre componentes sem passar **props** por cada nível da árvore. Imagine que seria impossível passar **user** por 10 componentes diferentes manualmente.

useEffect(fn, []) — roda uma vez quando o componente monta. O `[]` vazio significa "não re-execute quando nada mudar". Aqui, verifica se o usuário já estava logado (token no localStorage).

useAuth() — hook customizado que qualquer componente usa para acessar o usuário logado ou as funções de login/logout.

frontend/src/components/PrivateRoute.tsx

```
export function PrivateRoute({ children }) {  
  const { isAuthenticated } = useAuth();  
  return isAuthenticated ? <>{children}</> : <Navigate to="/login" replace />;  
}
```

O que é: Protetor de rotas. Se não está logado, redireciona para `/login`.

`<Navigate replace />` — redireciona sem adicionar a rota atual ao histórico do navegador. Sem `replace`, o botão "voltar" levaria de volta à rota protegida, gerando um loop.

frontend/src/pages/LoginPage.tsx

```
const handleSubmit = async (e: FormEvent) => {  
  e.preventDefault(); // impede o comportamento padrão do form (recarregar a  
  página)  
  setError('');  
  setLoading(true);  
  try {  
    await login(email, password);  
    navigate('/dashboard');  
  } catch (err) {  
    setError(err.response?.data?.error || 'Erro ao fazer login.');  } finally {  
    setLoading(false); // sempre executa, com ou sem erro  
  }  
};
```

O que é: Tela de login com formulário controlado.

`e.preventDefault()` — formulários HTML por padrão recarregam a página ao serem submetidos. Precisamos impedir isso para usar o React.

`finally` — bloco que sempre executa, independente de sucesso ou erro. Útil para limpar estados de carregamento.

`err.response?.data?.error` — encadeamento opcional (`?.`). Se `err.response` for `undefined`, não lança erro — retorna `undefined` e usa o fallback `'Erro ao fazer login.'`

frontend/src/pages/DashboardPage.tsx

```
const [tasks, setTasks] = useState<Task[]>([]);  
const [filter, setFilter] = useState<TaskFilter>('all');  
  
const filteredTasks = tasks.filter((t) => {  
  if (filter === 'pending') return !t.completed;  
  if (filter === 'completed') return t.completed;  
});
```

```
    return true; // 'all'
  });

  const handleComplete = async (id) => {
    const updated = await taskService.complete(id);
    setTasks((prev) => prev.map((t) => t.id === updated.id ? updated : t));
  };
}
```

O que é: Tela principal — lista, filtra e gerencia tarefas.

Filtro local — o filtro acontece no frontend, sem nova requisição ao backend. Os dados já estão em memória (`tasks`), então apenas filtramos o array. Isso é mais rápido para o usuário.

`setTasks(prev => prev.map(...))` — padrão importante do React. Após uma atualização, não re-buscamos todos os dados do servidor. Em vez disso, atualizamos apenas o item modificado no array local. Mais eficiente e sem "piscar" na tela.

frontend/src/components/TaskCard.tsx

```
<button
  onClick={() => !task.completed && onComplete(task.id)}
  disabled={task.completed}
  aria-label={task.completed ? 'Tarefa concluída' : 'Marcar como concluída'}
>
```

aria-label — acessibilidade. Leitores de tela (para pessoas com deficiência visual) leem esse texto para descrever o botão, já que o botão não tem texto visível.

`!task.completed && onComplete(task.id)` — curto-circuito: se `task.completed` for `true`, a segunda parte não executa. Evita completar uma tarefa que já está completa.

frontend/src/__tests__/LoginPage.test.tsx

```
it('chama login com os dados preenchidos', async () => {
  mockLogin.mockResolvedValueOnce(undefined);

  fireEvent.change(screen.getByPlaceholderText('seu@email.com'), {
    target: { value: 'test@test.com' },
  });
  fireEvent.click(screen.getByRole('button', { name: /entrar/i }));

  await waitFor(() => {
    expect(mockLogin).toHaveBeenCalledWith('test@test.com', '123456');
  });
});
```

O que é: Teste de componente React com Testing Library.

Filosofia da Testing Library: Teste como o usuário interage, não como o código é implementado. Use `getByPlaceholderText`, `getByRole` — não IDs internos ou classes CSS.

`mockLogin.mockResolvedValueOnce` — substitui a função real de login por uma fake que retorna sucesso. Isso isola o teste: estamos testando o componente, não a API.

`waitFor` — aguarda até que a asserção seja verdadeira. Necessário porque o formulário é assíncrono (tem `async/await`).

4. Docker — Como Tudo se Conecta

backend/Dockerfile

```
FROM node:20-alpine          # imagem base leve do Node.js

WORKDIR /app                  # pasta de trabalho dentro do container

COPY package*.json ./         # copia só o package.json primeiro...
RUN npm ci --only=production  # ...para aproveitar o cache do Docker

COPY src ./src                # copia o código fonte

EXPOSE 3000                   # documenta que a porta 3000 será usada
CMD ["node", "src/index.js"] # comando que inicia o servidor
```

Por que copiar `package.json` antes do código? O Docker cacheia cada passo. Se o código mudou mas as dependências não, ele pula o `npm ci` (passo lento) e vai direto para o `COPY src`. Economiza minutos em cada build.

alpine — versão minimalista do Linux. Imagem menor = deploy mais rápido e menos superfície de ataque.

npm ci — instala exatamente as versões do `package-lock.json`. Mais confiável que `npm install` para ambientes de produção.

frontend/Dockerfile

```
# ESTÁGIO 1: Build
FROM node:20-alpine AS builder
ARG VITE_API_URL=http://localhost:3000/api
ENV VITE_API_URL=$VITE_API_URL
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build          # gera a pasta dist/ com HTML/CSS/JS otimizados

# ESTÁGIO 2: Servir
```

```
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html # só copia o dist/
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Multi-stage build — técnica avançada do Docker. O estágio **builder** instala Node.js, todas as dependências de desenvolvimento e compila o projeto. O estágio final pega **apenas** os arquivos gerados (**dist/**) e os serve com Nginx. A imagem final não contém Node.js, npm, ou código fonte — apenas HTML/CSS/JS estáticos. Resultado: imagem de ~25MB em vez de ~500MB.

Nginx — servidor web ultra-leve, ideal para servir arquivos estáticos. No frontend, ele serve os arquivos e redireciona todas as rotas para **index.html** (necessário para o React Router funcionar).

`docker-compose.yml` (raiz)

```
services:
  postgres:
    image: postgres:15-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready"]
      interval: 5s
      retries: 5

  backend:
    build: ./backend
    depends_on:
      postgres:
        condition: service_healthy # espera o banco estar pronto

  frontend:
    build: ./frontend
    depends_on:
      - backend
```

O que é: Orquestrador que sobe todos os serviços com um único comando.

depends_on com condition: service_healthy — sem isso, o backend tentaria conectar ao banco antes dele estar pronto, falhando. O healthcheck verifica se o PostgreSQL está aceitando conexões antes de iniciar o backend.

Rede interna: Os containers se comunicam pelo nome do serviço. O backend usa **DB_HOST=postgres** — o Docker resolve **postgres** para o IP interno do container do banco.

5. CI/CD — GitHub Actions

`.github/workflows/ci.yml`

```
on:
  push:
    branches: [main]      # executa ao fazer push na main
  pull_request:
    branches: [main]      # executa ao abrir PR para a main

jobs:
  backend-tests:
    services:
      postgres:            # sobe um banco PostgreSQL real para os testes
        image: postgres:15-alpine

    steps:
      - uses: actions/checkout@v4      # baixa o código
      - uses: actions/setup-node@v4    # instala o Node.js
      - run: npm ci                    # instala dependências
      - run: npm test -- --coverage    # roda os testes com cobertura

  frontend-tests:
    steps:
      - run: npm run test              # Vitest
      - run: npm run build              # verifica se o build de produção funciona
```

O que é: Pipeline automatizado que roda os testes toda vez que código é enviado ao GitHub.

Por que é importante: Garante que ninguém (nem você) quebre o projeto acidentalmente. Se os testes falharem, o GitHub bloqueia o merge.

actions/checkout@v4 — action oficial que baixa o código do repositório para a máquina virtual do GitHub.

cache: npm — guarda o cache do **node_modules** entre execuções. Economiza ~1 minuto por job.

6. Fluxo Completo de uma Requisição

Vamos acompanhar o que acontece quando um usuário cria uma tarefa:

1. Usuário preenche o formulário e clica em "Criar"
└─ TaskForm.tsx → onSubmit()
2. Frontend chama o service
└─ taskService.create({ title, description })
3. Axios envia a requisição HTTP
└─ POST http://localhost:3000/api/tasks
└─ Header: Authorization: Bearer eyJhbGc... ← interceptor adicionou
4. Express recebe e roteia
└─ taskRoutes.js: router.post('/', verificarToken, taskController.create)

5. Middleware verifica o token
 - └─ `authMiddleware.js` → `jwt.verify(token, JWT_SECRET)`
 - └─ injeta `req.userId = "uuid-do-usuario"`
6. Controller valida e delega
 - └─ `taskController.js` → `taskService.criarTarefa(title, desc, userId)`
7. Service executa o SQL
 - └─ `INSERT INTO tasks (...) VALUES (...) RETURNING *`
 - └─ retorna a nova tarefa
8. Controller responde
 - └─ `res.status(201).json({ task: novaTarefa })`
9. Frontend recebe e atualiza a tela
 - └─ `setTasks(prev => [task, ...prev])`
 - └─ A nova tarefa aparece no topo da lista sem recarregar a página

Conceitos para Revisar

Conceito	Onde aparece	Resumo
JWT	<code>authController</code> , <code>authMiddleware</code>	Token assinado que prova identidade
<code>bcrypt</code>	<code>userService</code>	Hash seguro para senhas
Pool de conexões	<code>database.js</code>	Reutilização de conexões com o banco
SQL Injection	Toda query com <code>\$1</code>	Parâmetros separados da query = seguro
Middleware	<code>authMiddleware</code> , <code>cors</code> , <code>json</code>	Funções no meio do caminho <code>req→res</code>
Context API	<code>AuthContext</code>	Estado global sem prop drilling
Interceptors	<code>api.ts</code>	Lógica automática em toda requisição
Multi-stage build	<code>frontend/Dockerfile</code>	Build e produção em imagens separadas
Healthcheck	<code>docker-compose.yml</code>	Garante que serviços esperem uns pelos outros
CI/CD	<code>.github/workflows/ci.yml</code>	Testes automáticos a cada push

Para converter em PDF: abra este arquivo no VS Code, pressione `Ctrl+Shift+P`, digite "Markdown: Open Preview", depois `Ctrl+P` → "Print" → "Save as PDF".